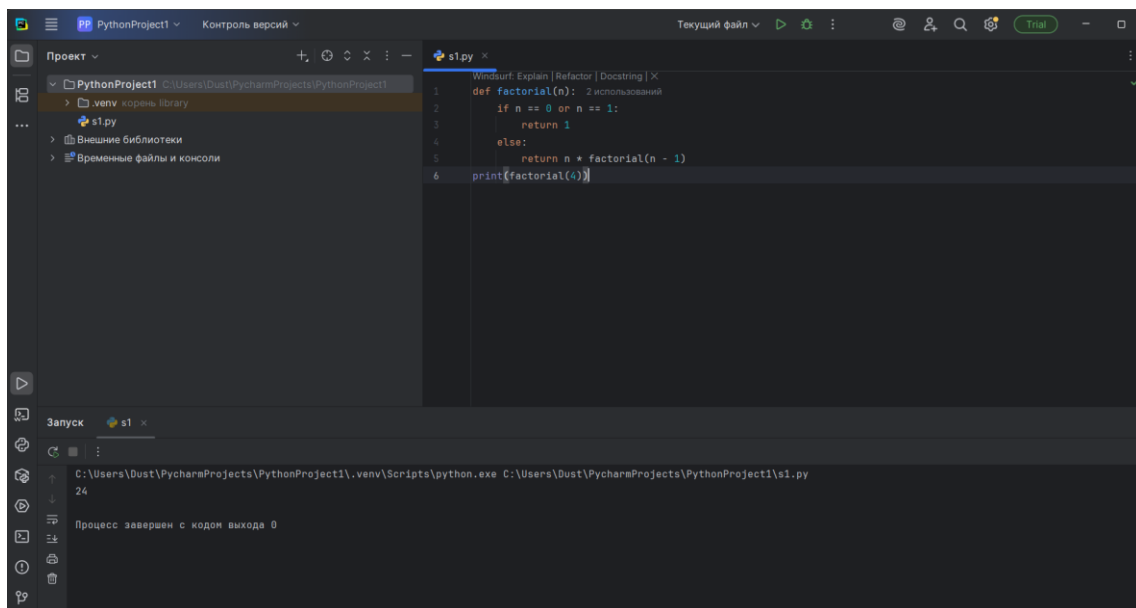


Задание 1

Задание 1. Простая рекурсия

Напишите рекурсивную функцию для вычисления факториала числа n .

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    else:  
        return n * factorial(n - 1)  
  
print(factorial(5)) # Результат: 120
```



Объяснение кода

Функция вычисляет **факториал числа** через рекурсию.

Как работает:

- Если $n = 0$ или 1 , возвращает 1 (базовый случай)
- Иначе возвращает $n \times \text{factorial}(n-1)$ (рекурсивный вызов)

Для $\text{factorial}(4)$:

- $4 \times 3 \times 2 \times 1 = 24$

Программа выводит: 24

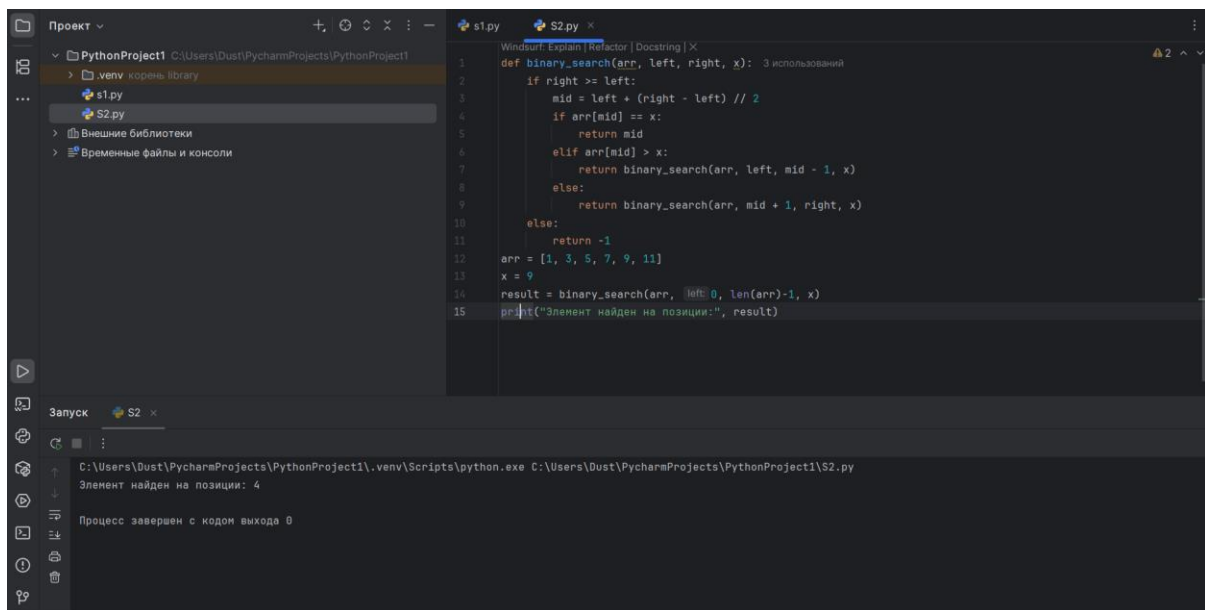
Задание 2

Задание 2. Рекурсивный поиск

Реализуйте рекурсивный алгоритм бинарного поиска элемента в отсортированном списке.

```
def binary_search(arr, left, right, x):
    if right >= left:
        mid = left + (right - left) // 2
        if arr[mid] == x:
            return mid
        elif arr[mid] > x:
            return binary_search(arr, left, mid - 1, x)
        else:
            return binary_search(arr, mid + 1, right, x)
    else:
        return -1

arr = [1, 3, 5, 7, 9, 11]
x = 9
result = binary_search(arr, 0, len(arr)-1, x)
print("Элемент найден на позиции:", result)
```



Бинарный поиск

Суть: Ищет элемент в отсортированном массиве, каждый раз деля диапазон пополам.

Алгоритм:

- Проверяет средний элемент
- Если больше искомого → ищет слева
- Если меньше → ищет справа
- Если не найден → возвращает -1

Результат: Находит 9 на позиции 4

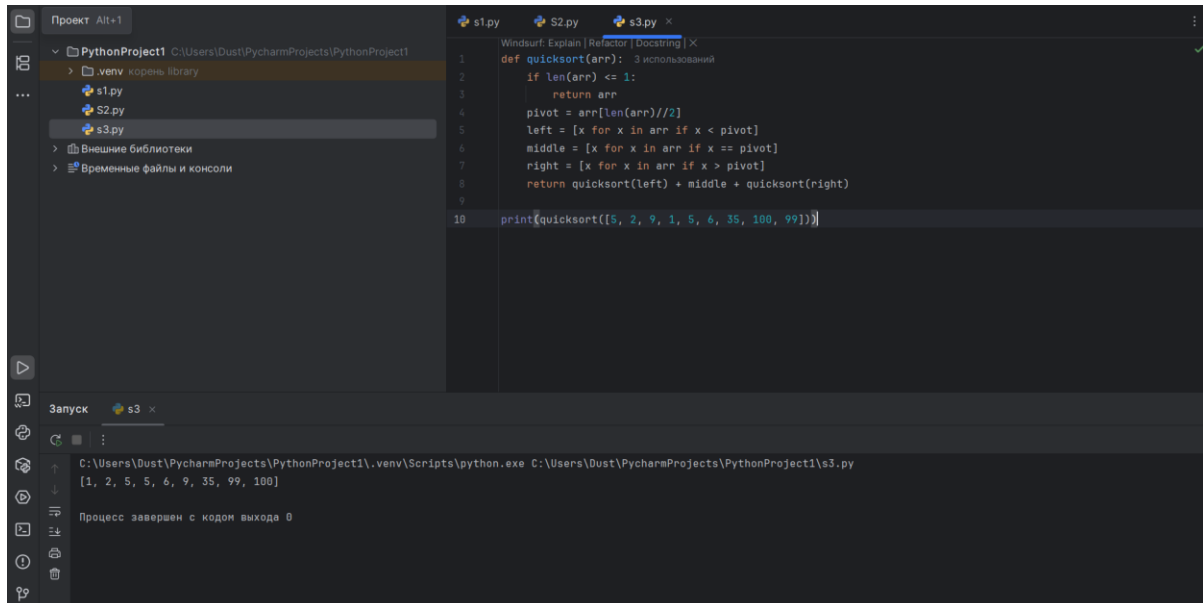
Задание 3

Задание 3. Рекурсивная сортировка (QuickSort)

Реализуйте алгоритм быстрой сортировки (QuickSort) с помощью рекурсии.

```
def quicksort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quicksort(left) + middle + quicksort(right)

print(quicksort([5, 2, 9, 1, 5, 6]))
```



Этот код реализует **быструю сортировку (quicksort)**:

1. **Базовый случай:** если массив из 0-1 элемента — возвращаем как есть
2. **Выбор pivot:** берем средний элемент `arr[len(arr)//2]`
3. **Разделение:** делим массив на три части:
 - a. `left` — элементы меньше `pivot`
 - b. `middle` — элементы равные `pivot`
 - c. `right` — элементы больше `pivot`
4. **Рекурсия:** сортируем `left` и `right`, объединяем результат

Результат: `[1, 2, 5, 5, 6, 9, 35, 99, 100]`

Сложность: $O(n \log n)$ в среднем, $O(n^2)$ в худшем случае.

Ответы на вопросы

1. Что такое рекурсия и как она реализуется?

Рекурсия — это когда функция вызывает сама себя для решения задачи, разбивая её на более мелкие подзадачи.

```
def factorial(n):  
    if n <= 1:  
        return 1  
    return n * factorial(n - 1)
```

2. Что такое базовое условие рекурсии?

Базовое условие — это условие выхода из рекурсии, при котором функция возвращает результат без дальнейших вызовов. Без него возникнет бесконечная рекурсия и переполнение стека.

3. Чем итерация отличается от рекурсии?

- **Рекурсия:** функция вызывает себя, использует стек вызовов, нагляднее для сложных задач
- **Итерация:** использует циклы, расходует меньше памяти, обычно быстрее

Рекурсия

```
def sum_rec(n):  
    return 0 if n == 0 else n + sum_rec(n - 1)
```

Итерация

```
def sum_iter(n):  
    result = 0  
    for i in range(1, n + 1):  
        result += i  
    return result
```

4. Когда рекурсия неэффективна?

- **Повторяющиеся вычисления:** fibonacci(5) многократно пересчитывает одни и те же значения
- **Глубокая рекурсия:** риск переполнения стека
- **Простые задачи:** где итерация очевидна и проще

- **Отсутствие оптимизации хвостовых вызовов** в языке

5. Как оптимизировать рекурсию?

Мемоизация — кэширование результатов:

```
def fib_memo(n, memo={}):  
    if n in memo:  
        return memo[n]  
    if n <= 1:  
        return n  
    memo[n] = fib_memo(n-1, memo) + fib_memo(n-2, memo)  
    return memo[n]
```

Хвостовая рекурсия:

```
def factorial_tail(n, acc=1):  
    return acc if n <= 1 else factorial_tail(n - 1, n * acc)
```

Динамическое программирование — замена на итерацию с сохранением результатов

Правило: рекурсия хороша для деревьев, графов, алгоритмов «разделяй и властвуй». Для простых задач лучше итерация.